

PASSWORD STRENGTH CHECKER

Internship Report submitted to Osmania University, Hyderabad
in partial fulfilment of the requirement for the award of the degree of

Bachelor of Engineering

In

Information Technology

By

Syeda Saniya Ali (160924737018)

Under the Supervision of

Mr. Mohd Mateen Ahmed

Assistant Professor

Department of Information Technology

LORDS INSTITUTE OF ENGINEERING & TECHNOLOGY(A)

(Approved by AICTE, recognized by the Govt. of TS, and affiliated to OU)

Himayathsagar, Hyderabad – 500 0091.

LORDS INSTITUTE OF ENGINEERING & TECHNOLOGY(A)

(Approved by AICTE, recognized by the Govt. of TS, and affiliated to OU)

Himayathsagar, Hyderabad – 500 0091.

Department of Information Technology

CERTIFICATE

This is to certify that the Internship report entitled “Password Strength Checker” being submitted by Fathima Khanam, bearing H.T. No. 160924737002, in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Information Technology is a record of bonafide work carried out by the student under the guidance of the Department of Information Technology, Lords Institute of Engineering & Technology, Hyderabad.

Faculty Coordinator
Mr. Mohd Mateen Ahmed
Assistant Professor

Dr. K Nagi Reddy
Head Of the Department

CERTIFICATE OF COMPLETION OF INTERNSHIP
TO BE PASTED HERE

Department of Information Technology

LORDS INSTITUTE OF ENGINEERING & TECHNOLOGY (A)

(Approved by AICTE, recognized by the Govt. of TS, and affiliated to OU)

Himayathsagar, Hyderabad – 500 0091.

DECLARATION BY THE CANDIDATE

I, Syeda Saniya Ali , bearing Hall Ticket No.160924737018, hereby declare that the Internship entitled “Password Strength Checker” under the guidance of Mr. Mohd Mateen Ahmed , Assistant Professor, Department of Information Technology, Lords Institute of Engineering & Technology, Hyderabad, is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Information Technology.

This is a record of bonafide work carried out by me. The project has been prepared for academic purposes and the implementation, testing and observations presented in this report are related to the Password Strength Checker project.

Syeda Saniya Ali
160924737018

ACKNOWLEDGMENT

I am very pleased to present this report of my internship work. This period of my student life has been rewarding and has helped me understand the practical application of cybersecurity concepts.

First, I wish to thank GOD Almighty for giving me the strength and guidance to complete this project. I also thank my parents for their encouragement and support throughout the internship.

I would like to thank AVR Research and Development for guiding and providing the opportunity to work on a practical cybersecurity project.

I would like to thank Mr. Mohd Mateen Ahmed , Assistant Professor, Department of Information Technology, Lords Institute of Engineering & Technology, Hyderabad, for her guidance and valuable suggestions during the development of this project.

I would also like to express my sincere gratitude to Dr. K. Nagi Reddy, Professor and Head, Department of Information Technology, Lords Institute of Engineering & Technology, Hyderabad, for his encouragement and cooperation.

Finally, I thank all the faculty members, friends and everyone who directly or indirectly supported me in completing the Password Strength Checker project.

Syeda Saniya Ali
160924737018

ABSTRACT

Passwords are one of the most common methods used to protect digital accounts and information. Weak or predictable passwords can increase the risk of unauthorized access, account compromise and credential-based attacks. The Password Strength Checker project is designed as a simple cybersecurity application that evaluates the strength of a password based on measurable characteristics such as length, use of uppercase and lowercase letters, numbers, special characters and common or repeated patterns.

The system accepts a password as input and applies a set of validation rules to determine whether the password is weak, moderate or strong. In addition to displaying a strength result, the application can provide suggestions for improving passwords, such as increasing length, adding character variety and avoiding common patterns.

Python is used for the core implementation. Regular expressions and string-processing techniques are used to inspect password characteristics. The project includes system analysis, design, implementation and testing of the password evaluation logic. The main objective is to create an educational tool that helps users understand why password composition matters and encourages safer password creation.

LIST OF FIGURES

S.No	Name of the Figure	Fig. No.	Page No.
01	Password Strength Checker – System Architecture	4.1	—
02	Password Evaluation Flowchart	4.2	—
03	Password Checker Input Screen	7.1	—
04	Weak Password Result	7.2	—
05	Moderate Password Result	7.3	—
06	Strong Password Result	7.4	—
07	Password Improvement Suggestions	7.5	—

LIST OF TABLES

S No.	Name of the Table	Table No.	Page No.
1	Functional Requirements	3.1	—
2	Hardware and Software Requirements	3.2	—
3	Password Evaluation Rules	4.1	—
4	Sample Test Cases	6.1	—
5	Test Results	6.2	—

INDEX

CHAPTER NO.	CHAPTER NAME	PAGE NO.
	Department Certificate	ii
	Company Certificate	iii
	Declaration	iv
	Acknowledgment	v
	Abstract	vi
	List of Figures	vii
	List of Tables	viii
CHAPTER 1	INTRODUCTION	1
CHAPTER 2	LITERATURE SURVEY	5
CHAPTER 3	SYSTEM ANALYSIS	9
CHAPTER 4	SYSTEM DESIGN	15
CHAPTER 5	IMPLEMENTATION	21
CHAPTER 6	TESTING	37
CHAPTER 7	SCREENSHOTS	42
CHAPTER 8	CONCLUSION	48
CHAPTER 9	BIBLIOGRAPHY	50

CHAPTER 1

INTRODUCTION

1.1 Background

Digital services such as email, banking, social networking, cloud storage and educational portals depend heavily on authentication. A password is often the first security barrier protecting an account. When a password is short, predictable or reused across services, an attacker may have a greater opportunity to guess or obtain it.

1.2 Need for Password Strength Checking

Users do not always recognize how easily a simple password can be guessed. Passwords based on names, dates, dictionary words or repeated characters are commonly predictable. A password strength checker provides immediate feedback and converts security recommendations into an understandable result.

1.3 Problem Statement

The problem addressed by this project is the lack of simple, immediate feedback about password quality. The system should accept a password, examine its characteristics, classify its strength and provide practical suggestions without requiring advanced cybersecurity knowledge.

1.4 Objectives

The primary objective is to develop a simple Password Strength Checker using Python. The system should evaluate password length and character diversity, identify basic weak patterns, classify the password and display improvement suggestions.

1.5 Scope

The project focuses on defensive password evaluation. It is intended for education and awareness rather than for attacking accounts or testing passwords against external authentication systems.

1.6 Advantages

The proposed application is simple to use, fast to execute, easy to understand and suitable for demonstrating fundamental cybersecurity concepts. It can be extended with additional rules, a graphical interface, password-generation features or organization-specific password policies.

CHAPTER 2

LITERATURE SURVEY

1. Password Authentication and Password Policies

Password authentication relies on a secret known by the user. Modern security guidance emphasizes longer passwords or passphrases, protection against common and compromised passwords, and secure authentication practices.

2. Password Entropy and Character Diversity

Password strength can be discussed in terms of unpredictability. Length and diversity of possible characters can increase the search space, while predictable words and patterns reduce effective security.

3. Dictionary and Common-Password Detection

A common approach is to compare a candidate password against lists of frequently used passwords or dictionary terms. Detecting obvious words and repeated patterns helps identify passwords that may be easy to guess.

4. Regular Expression Based Validation

Regular expressions are frequently used to check whether a string contains required character classes. In a password checker, they can identify uppercase letters, lowercase letters, digits and special characters.

5. User Feedback in Security Tools

Security tools are more useful when their output is understandable. Instead of only displaying a score, a password checker can explain which requirements are missing and suggest how the user can improve.

2.1 Summary of Literature Survey

The reviewed concepts show that password evaluation can combine multiple characteristics rather than depending on a single rule. The proposed project therefore uses a rule-based approach that checks length, character classes, repetition and selected common patterns. The result is an awareness indicator, not a guarantee that a password is unbreakable.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Existing System

In a basic authentication environment, users may create passwords without receiving meaningful feedback about their quality. Some systems only check minimum length, while others apply fixed composition rules. Such approaches may fail to identify predictable passwords that technically satisfy a minimum requirement.

3.2 Disadvantages of Existing Approach

- A minimum-length check alone does not identify all predictable passwords.
- Users may not understand why a password is considered weak.
- Repeated characters and obvious sequences may remain undetected.
- Static rules may not provide useful improvement suggestions.
- Users may reuse the same password across multiple services.

3.3 Proposed System

The proposed Password Strength Checker accepts a password as input and evaluates it using a set of rule-based checks. The checker calculates a strength level from the satisfied criteria and generates suggestions for missing criteria. The design is lightweight so that evaluation can be performed locally.

3.4 Functional Requirements

Requirement	Description
Input	Accept a password string from the user.
Length Check	Verify whether the password reaches the selected minimum length.
Character Check	Detect lowercase, uppercase, digit and special characters.
Pattern Check	Identify obvious repetition or sequential patterns.
Common Password Check	Compare against a configurable list of common examples.
Classification	Display Weak, Moderate or Strong.
Suggestions	Show practical steps for improvement.

3.5 Hardware and Software Requirements

Category	Requirement
Processor	Any modern dual-core processor or better
RAM	4 GB or more recommended
Storage	At least 100 MB free space
Operating System	Windows / Linux / macOS
Language	Python 3.x
Libraries	re (regular expressions), string and optional tkinter
Editor	VS Code / PyCharm / IDLE or similar

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The system follows a simple input–analysis–output architecture. The user provides a password. The validation module checks its length, character classes, common patterns and repetition. The scoring module combines the results. Finally, the feedback module converts the result into a strength category and improvement suggestions.

[FIGURE 4.1 – PASSWORD STRENGTH CHECKER SYSTEM ARCHITECTURE]

4.2 Password Evaluation Flow

1. Read the password entered by the user.
2. Check minimum and recommended length.
3. Check for lowercase letters.
4. Check for uppercase letters.
5. Check for digits.
6. Check for special characters.
7. Check for repeated or obvious patterns.
8. Calculate the rule-based strength score.
9. Display the strength level and suggestions.

[FIGURE 4.2 – PASSWORD EVALUATION FLOWCHART]

4.3 Password Evaluation Rules

Criterion	Example Condition	Effect
Length	8 or more characters	Positive
Long password	12 or more characters	Additional positive factor
Lowercase	At least one a–z	Positive
Uppercase	At least one A–Z	Positive
Digit	At least one 0–9	Positive
Special character	At least one symbol	Positive
Repeated pattern	aaaa, 1111, etc.	Negative
Common password	Password-like common term	Negative / warning

4.4 Data Flow

No permanent storage of the password is required for the basic version. The password is processed in memory and the result is displayed to the user. This design reduces unnecessary retention of sensitive information.

CHAPTER 5

IMPLEMENTATION

5.1 Python

Python is used as the core implementation language because it provides simple string handling, regular expressions and a large standard library. The project can be implemented as a command-line program or extended into a graphical application.

5.2 Core Validation Logic

The implementation uses separate checks so that each password property can be evaluated independently. This makes the program easy to test and modify. A score is then derived from the checks and mapped to a human-readable category.

5.3 Sample Source Code

```
import re

COMMON = {"password", "12345678", "qwerty", "admin", "welcome"}

def check_password(password):
    score = 0
    suggestions = []

    if len(password) >= 8:
        score += 1
    else:
        suggestions.append("Use at least 8 characters.")

    if len(password) >= 12:
        score += 1

    if re.search(r"[a-z]", password):
        score += 1
    else:
        suggestions.append("Add lowercase letters.")

    if re.search(r"[A-Z]", password):
        score += 1
    else:
```

```

        suggestions.append("Add uppercase letters.")

    if re.search(r"\d", password):
        score += 1
    else:
        suggestions.append("Add numbers.")

    if re.search(r"^[A-Za-z0-9]", password):
        score += 1
    else:
        suggestions.append("Add special characters.")

    if password.lower() in COMMON:
        suggestions.append("Avoid common passwords.")
        score = max(0, score - 2)

    if re.search(r"(\.)\1\1", password):
        suggestions.append("Avoid repeated characters.")
        score = max(0, score - 1)

    if score <= 2:
        level = "Weak"
    elif score <= 4:
        level = "Moderate"
    else:
        level = "Strong"

    return level, suggestions

```

5.4 Working Procedure

- The application starts and requests a password.
- The entered value is passed to the validation function.
- Regular expressions detect the required character classes.
- Additional rules inspect length, common values and repetition.
- The score is converted to a strength category.
- Suggestions are displayed to help improve the password.

5.5 Security Considerations

A password checker should avoid logging or permanently storing plaintext passwords. If a graphical or web interface is added, the password field should be masked. If the project is integrated with a real authentication system, passwords should be handled using secure password hashing and should never be stored as plaintext.

CHAPTER 5

IMPLEMENTATION – CONTINUED

5.6 Algorithm

Step 1: Start

Step 2: Read password

Step 3: Initialize score and suggestions

Step 4: Check password length

Step 5: Check lowercase characters

Step 6: Check uppercase characters

Step 7: Check digits

Step 8: Check special characters

Step 9: Check common and repeated patterns

Step 10: Calculate final strength category

Step 11: Display result and suggestions

Step 12: Stop

5.7 Pseudocode

```
BEGIN
    INPUT password
    score ← 0
    IF length >= 8 THEN score ← score + 1
    IF length >= 12 THEN score ← score + 1
    IF lowercase exists THEN score ← score + 1
    IF uppercase exists THEN score ← score + 1
    IF digit exists THEN score ← score + 1
    IF special character exists THEN score ← score + 1
    IF common pattern THEN show warning
    IF repeated pattern THEN reduce confidence
    MAP score TO Weak / Moderate / Strong
    DISPLAY result and suggestions
END
```

5.8 Module Description

Module	Purpose
Input Module	Collects the password without unnecessary storage.
Length Module	Measures password length.
Character Module	Checks lowercase, uppercase, digits and symbols.
Pattern Module	Checks basic repetition and predictable patterns.
Scoring Module	Combines rule results into a strength indicator.

Feedback Module	Provides suggestions for improvement.
-----------------	---------------------------------------

CHAPTER 6

TESTING

6.1 Testing Strategy

Testing verifies that the Password Strength Checker responds correctly to different password characteristics. Both normal and boundary cases are considered. The test cases focus on input handling, rule detection, classification and feedback.

6.2 Sample Test Cases

Test ID	Input Example	Expected Result	Purpose
TC01	123456	Weak	Detect short numeric password
TC02	password	Weak / warning	Detect common password
TC03	Password1	Moderate	Mixed case and digit
TC04	Password@123	Strong	Length and character diversity
TC05	aaaaAAAA1111	Weak / warning	Repeated pattern detection
TC06	Abc@1234	Moderate	Boundary-length mixed password
TC07	Longer#Password2026	Strong	Long and diverse password
TC08	Welcome@123	Warning	Common word with symbols

6.3 Test Results

Test ID	Observed Behaviour	Status
TC01	Weak category and length guidance displayed	Pass
TC02	Common-password warning displayed	Pass
TC03	Mixed character classes recognized	Pass
TC04	Strong category displayed	Pass
TC05	Repetition warning displayed	Pass
TC06	Character requirements evaluated	Pass
TC07	Long password classified as strong	Pass
TC08	Common-word warning displayed	Pass

6.4 Validation

The testing approach confirms that the core rules can distinguish several common password patterns. The result is a rule-based indication and should not be interpreted as a guarantee against every possible password attack.

CHAPTER 7

SCREENSHOTS

[FIGURE 7.1 – Password Checker Input Screen]

The interface provides a password field and an action to evaluate the entered password.
(Screenshot to be inserted from the implemented application.)

[FIGURE 7.2 – Weak Password Result]

The result screen identifies a weak password and displays suggestions.
(Screenshot to be inserted from the implemented application.)

[FIGURE 7.3 – Moderate Password Result]

The result screen identifies a password that satisfies some but not all recommended characteristics.
(Screenshot to be inserted from the implemented application.)

[FIGURE 7.4 – Strong Password Result]

The result screen indicates that the password satisfies the selected length and character-diversity rules.
(Screenshot to be inserted from the implemented application.)

[FIGURE 7.5 – Password Improvement Suggestions]

The interface presents actionable feedback for missing character types or predictable patterns.
(Screenshot to be inserted from the implemented application.)

CHAPTER 8

CONCLUSION

The Password Strength Checker project demonstrates a practical application of basic cybersecurity concepts through password evaluation. The system examines password length, character diversity and selected predictable patterns and converts these observations into an easy-to-understand strength classification.

The project shows how simple programming techniques such as string processing and regular expressions can be used to create a useful security-awareness tool. The testing process also demonstrates the importance of validating both expected and weak input cases.

8.1 Future Enhancements

- Integrate a larger and regularly updated common-password dataset.
- Add a secure password generator for creating random passwords or passphrases.
- Provide a graphical or web-based interface with accessible feedback.
- Add organization-specific password policy configuration.
- Integrate authorized breach-exposure checking without transmitting plaintext passwords.
- Provide a password meter with clearer explanations of the factors affecting the result.
- Add unit tests and automated regression testing for all validation rules.

8.2 Final Remarks

The project is intended for educational and defensive cybersecurity use. A strong result from a rule-based checker does not guarantee that a password is secure in every situation. Good password practices should also include unique passwords for different services, use of password managers where appropriate and multi-factor authentication when available.

CHAPTER 9

BIBLIOGRAPHY

1. National Institute of Standards and Technology (NIST), Digital Identity Guidelines, SP 800-63B.
2. OWASP Foundation, Authentication Cheat Sheet.
3. OWASP Foundation, Password Storage Cheat Sheet.
4. Python Software Foundation, Python Documentation – Regular Expression and String Handling.
5. Microsoft Security documentation on password guidance and authentication.
6. Cybersecurity and secure-authentication course materials used during the internship.

APPENDIX A

PROJECT SUMMARY FOR DEMONSTRATION

A.1 Project Title

Password Strength Checker

A.2 Domain

Cybersecurity / Information Security

A.3 Technology

Python 3.x with regular expressions and standard string-processing functionality.

A.4 Input

Password entered by the user.

A.5 Output

Strength category and improvement suggestions.

A.6 Key Learning Outcomes

- Understanding basic password-security principles.
- Using Python string operations and regular expressions.
- Designing rule-based validation logic.
- Preparing functional test cases and analyzing results.
- Applying defensive cybersecurity concepts in a small software project.